# Lab Assignments
# Computer Graphics DT519A,
# Örebro University, autumn 2024

November 19, 2024

## 3 Mapping techniques

### Learning goals

- Getting acquainted with mapping techniques for textures and reflections, in general.
- Managing texture mapping in OpenGL.
- Managing environment mapping in OpenGL.
- Optionally: bump mapping.
- Optionally: shadow mapping.

### Mandatory 3.1 Texture mapping

In this lab, you will model a 20-sided die (an *icosahedron*) and endow it with more and more detail. In the first task, you will add a texture to draw numbers on each of the 20 sides. (Formally speaking, the texture will modify the *albedo*; i e, the "colour" in your BRDF; of the object.)

In addition to these instructions, you may also want to refer to the OpenGL wiki on texture mapping.[1]

The skeleton code for `lab3-1.cpp` contains the geometry for the icosahedron. Copy this geometry into your previous code from Task 2.1 where you made a six-sided die and computed its normals. Using the fragment shader from 2.1 (colouring based on the normal direction), it should look like Figure 1.

Your next task is to generate an OpenGL texture object, using the following lines of code:

```
1 GLuint texture_handle;
2 glGenTextures( 1, &texture_handle );
3 glActiveTexture( GL_TEXTURE0 );
4 glUniform1i( glGetUniformLocation(shader_program, "tex_sampler"), 0 );
5 glBindTexture( GL_TEXTURE_2D, texture_handle );
```

What this does is to create a new *texture object* (with no actual content) that is referenced by `texture_handle`, and binds it (sets it to the currently active texture object). Line 2 additionally activates *texture unit* nr 0. In order to use multiple textures (which we will do in the next task), we will have to use separate GPU texture units.

In your fragment shader, create a `uniform sampler2D tex_sampler`[2] variable, which it will use to access the texture from that texture unit on the graphics card. You'll need to set the value of `tex_sampler` to number of the texture unit; i e, 0 for `GL_TEXTURE0`. This is what happens on line 4.

[1] https://www.opengl.org/wiki/Texture
[2] https://www.opengl.org/wiki/Sampler_(GLSL)



Figure 1: Icosahedron with fragment shader based on surface normals.

Now load an image to the texture. For this purpose you can use the function `lodepng_decode32_file()` available from `lodepng.h` (given as part of the lab code base).

```
unsigned char* image_data;
unsigned image_w;
unsigned image_h;
unsigned err_code =
  lodepng_decode32_file( &image_data, &image_w,  &image_h,
                         "../../common/data/numberline_hires.png" );
if (!err_code)
  std::cout << "Read " << image_h << " x " << image_w << " image\n";
```

Now that you have loaded an image on the CPU side, you need to send it to the graphics card. This is done with the `glTexImage2D` function. The *target* should be `GL_TEXTURE_2D` (since that what you just bound above), the *level* should be zero, the *internalformat* and *format* is `GL_RGBA` for the image `numberline.png` that you just loaded, the *width* and *height* are available from the lodepng call above, *border* is always(!) zero, and the image data is of type `GL_UNSIGNED_BYTE` (which matches the `unsigned char* image_data` that stores the image data from lodepng.

Using the `glTexParameterXXX`[3] functions, a number of parameters can be set to determine how the texture image is treated. E g, what happens if you try to access a texel that is outside the dimensions of the image? Look up the description of `GL_TEXTURE_WRAP_*` in the docs. (This is an example of what RTR4 book calls "corresponder function" in Chapter 6.1.2.)

A caveat at this point is that the default texture sampling parameters might result in you getting no visible texture at all. Two terms related to MIP/RIP mapping are *texture minification* and *magnification*. How to sample a value from the texture is governed by `glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, *)` and `GL_TEXTURE_MAG_FILTER`. The default value of `GL_TEXTURE_MIN_FILTER` is `GL_NEAREST_MIPMAP_LINEAR`, which requires that a MIP map has been generated. So you should either call `glGenerateMipMap` to do that, or select another sampling parameter that doesn't require a MIP map (such as nearest-neighbour or linear interpolation). You can read more in the `glTexParameter` docs.

[3] https://www.opengl.org/sdk/docs/man/html/glTexParameter.xhtml

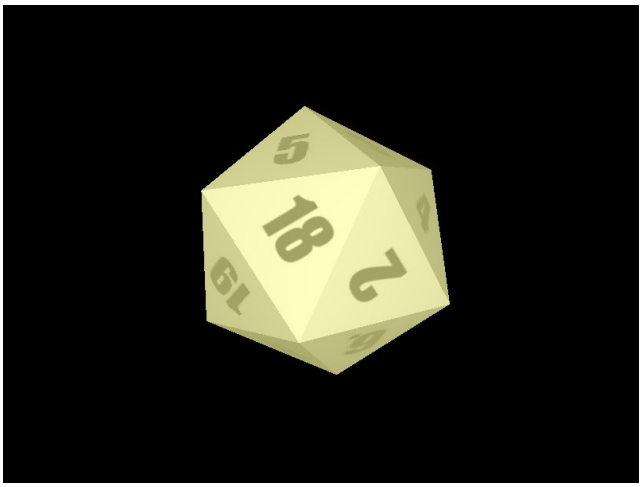



Figure 2: A textured 20-sided die.

You are now almost ready to start performing texturing inside your fragment shader. But first you need to decide which part of the texture images should correspond to each fragment that you are drawing. For that, we need a new vertex attribute: the $uv$ texture coordinates of each vertex.

Load the 2D `uv_coords` array in the skeleton code into a VBO, similar to how you have already done with positions and normals. (The $uv$ coordinates specified in the code depend on having the same order of the `points` vector as in the skeleton code.)

The vertex shader then should have a new vertex attribute `in vec2 texcoord` to store these texture coordinates. Let the vertex shader pass along the texture coordinates unchanged to the fragment shader in a new `out` variable. (Because transformations in the vertex shader should not affect the texture coordinates of the vertices!)

Finally, you should add the fragment shader code that will actually fetch the data from the texture units and do something interesting with it. You can do this by adding:

```
vec4 tex = texture(tex_sampler, uv_coords);
```

The `texture` function picks a sample at point `uv_coords` from the texture sampler.

Now you can use the `tex` variable in your lighting computation. For example, try setting the fragment colour to `tex.rgb`, or the roughness of a Cook–Torrance BRDF using `tex.r`.

You should now have textured 20-sided die with a number on each side, as in Figure 2.

Mandatory

## 3.2 Environment mapping

The goal of this task is to simulate a reflective (mirror-like) material by using an environment map. See Figure 3. There are images that you can use for your environment map under `common/data/`. See `cube_back.png`, `cube_front.png`, etc. You should use these images as a texture cube in OpenGL, from which a sample can be taken for the reflected view vector, which will then add to the colour of the surface, and make it appear as if it is more shiny.

1. Generate a new texture with `glGenTextures`.
2. Activate a new texture unit `GL_TEXTURE1`.
3. Tell the shader program which texture unit the environment map is in, by declaring a new `uniform samplerCube env_sampler` and passing the correct texture unit number with a call to `glUniform1i`, as you did before. (The `samplerCube` picks texels from a cube map, given a 3D vector, as described in Lecture 5. So you don't have to implement the mapping from the 3D vector to the cube surface yourself.)
4. Bind your texture as a `GL_TEXTURE_CUBE_MAP` (instead of `GL_TEXTURE_2D`).
5. Similar as in Task 3.1, you should also set the `GL_TEXTURE_MIN/MAG_FILTER` parameters of the `GL_TEXTURE_CUBE_MAP` to, e.g., `GL_LINEAR`.
6. Load the six sides of the cube map with `lodepng_decode32_file`.
7. Now you'll have to call `glTexImage2D` six times, to assign the six images to the texture unit. The *target* (i e, the first argument to `glTexImage2D`) should not be a `GL_TEXTURE_2D` this time. Instead, you should set it to `GL_TEXTURE_CUBE_MAP_[POSITIVE/NEGATIVE]_[X/Y/Z]` depending on which side of the cube it is. (Positive X is right, etc, and positive Z is back.)
8. Update your fragment shader to pick a colour from the environment map when computing the colour of a fragment.
   (a) In order to do so, you will need to compute the *reflected view direction*; i e, where in the environment does the light that gets specularly reflected into the camera come from? You can use the GLSL `reflect` function here.
   (b) Pick a colour from the environment map. This is again done using a call to the GLSL `texture` function, but using your reflected view vector: `texture(env_sampler, reflection_vector)`.
   (c) Mix it with the colour you computed in your BRDF. You can set a parameter $r \in [0, 1]$ and set the final colour to

   $$r \cdot (\text{colour from environment map}) + (1 - r) \cdot (\text{colour from Lab 2}).$$

   If you want to, you can implement a Fresnel parameter instead of using a constant $r$. You can implement Schlick's approximation of Fresnel: $r = F_0 + (1 - F_0)(1 - \cos\theta)^5$, with $F_0$ between zero and one (see Chapter 9.5.1 in the RTR book).

The result should look something like Figure 3. In order to better debug this, it may be easier to be able to rotate the object in very small increments. Otherwise the reflections can tend to jump around quite a lot. The teapot, with its smooth round shapes, is a good object to see the effect of the reflections well, in case you get confused about which image goes where in the cubemap.

## +1 point 3.3 Normal mapping

In addition to modifying the *material* properties with an image, you can also use images to cheaply simulate more complex *geometry*, by tweaking the normal vector of the otherwise flat surfaces of your object.



Figure 3: Mirror objects, from Task 3.2. The icosahedron, and an example of how the bunny (with Fresnel reflection) and teapot from Lab 2 might look.

Provided with the lab skeleton code (under `common/data`) is another image `numberline_nmap_hires.png`. Have a look at that image. It specifies normals as full 3D coordinates in the *tangent space* of the surface, scaled so that $[-1, 1]$ maps to $[0, 1]$. In other words, a normal vector that agrees with the actual normal of the surface would be $(0, 0, 1)$ in the tangent space, and maps to colour $(0.5, 0.5, 1)$ in the normal map. Your task now is to use that image as a normal map (bump map).

1. Load that image with `lodepng_decode32_file()` as you did with the texture map in Task 3.1.
2. Instantiate a new texture, using `glGenTextures` and `glBindTexture` as before.
3. Assign the normal map to the active texture using `glTexImage2D`.
4. Activate a new texture unit with `glActiveTexture`.
5. In your fragment shader, make a `uniform sampler2D normal_sampler` to read from the normal map.
6. From the CPU side, set the right number for the texture unit in that `uniform sampler2D`.
7. Set the mipmap parameters with `glTexParameteri` as before.

Now comes the tricky bit: specifying *tangent* and *bitangent* vectors (illustrated in Figure 4) as vertex attributes to be passed through the vertex shader to the fragment shader. First of all, declare two `std::vector<glm::vec4>` where you'll store the new vectors (just as you did with the normal vectors before.) Computing the tangent and bitangent vectors is best done in the same loop where you loop over the triangle faces of the object and compute the normals.

For each face, compute the difference in $uv$ and $xyz$ coordinates for two of its edges:

$$\vec{e_1} = \vec{p_2} - \vec{p_1}, \tag{1}$$
$$\vec{e_2} = \vec{p_3} - \vec{p_1}, \tag{2}$$
$$\vec{f_1} = \vec{u_2} - \vec{u_1} = (\Delta u_1, \Delta v_1), \tag{3}$$
$$\vec{f_2} = \vec{u_3} - \vec{u_1} = (\Delta u_2, \Delta v_2). \tag{4}$$

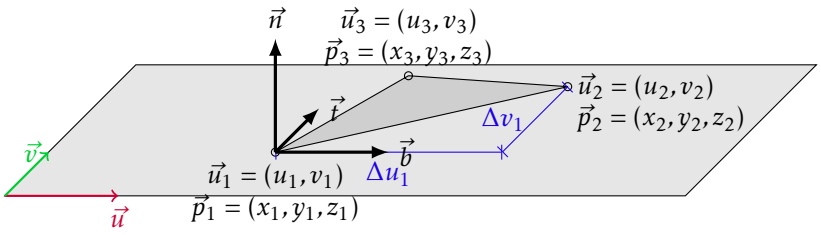



Figure 4: A triangle on a texture. Given the 2D $uv$ coordinates at its vertices, we also need to compute the 3D vectors $\vec{t}$ and $\vec{b}$ for each vertex.

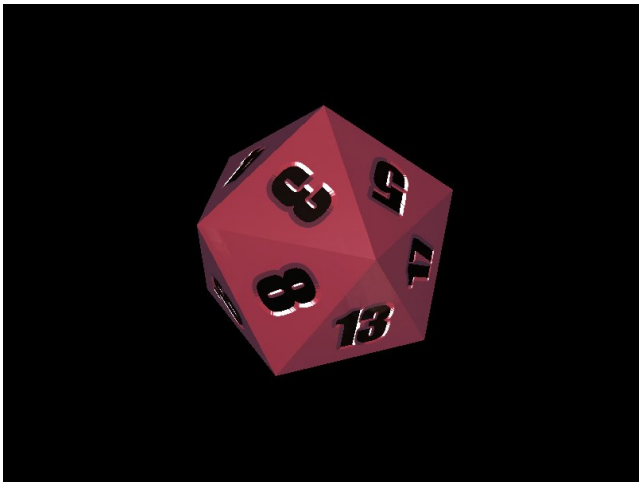


Figure 5: Now with a bump map! (And a different colour.)

Now, the tangent and bitangent can be computed as

$$r = (\Delta u_1 \Delta v_2 - \Delta v_1 \Delta u_2), \tag{5}$$
$$\vec{t} = (\Delta v_2 \vec{e}_1 - \Delta v_1 \vec{e}_2)/r, \tag{6}$$
$$\vec{b} = (\Delta u_1 \vec{e}_2 - \Delta u_2 \vec{e}_1)/r. \tag{7}$$

Normalise $\vec{t}$ and $\vec{b}$, and put the newly computed tangent and bitangent vectors in a VBO.

In the vertex shader, send the normal, tangent, and bitangent vectors (in view space, no perspective applied) to the fragment shader (just like you have done with the normal vectors before).

In the fragment shader, do the following.

1. Scale the vector that you get from your `normal_sampler` so that it is again on the interval $[-1, 1]$ instead of $[0, 1]$.

2. The normals in the supplied normal map make it appear as the numbers are outdented from the surface (bumps). If you would rather have them indented (cavities) you can negate the $xy$ (not $z$) coordinates of the normal vector here.

3. The three vectors $\vec{t}$, $\vec{b}$, $\vec{n}$ form an orthogonal base that can be used as a rotation matrix.

```
mat4 TBN = mat4(tangent, bitangent, normal, vec4(0,0,0,1));
```

Now TBN can be used to rotate the normal vector you get from the normal map to the view space, so that you can plug it in the lighting equations you have from before.

Alternatively, you can create TBN in the vertex shader and use the transpose of it to transform the light and view vectors into tangent space, rather than the other way around. This will lead to a little more work in the vertex shader, but less in the fragment shader (which is run more often) so will usually be more efficient — but it doesn't matter for this small scene!

The result should look something like Figure 5.

**+2 points**

## 3.4 Shadow mapping

Now it is time to add shadows, by means of *shadow mapping*, to the scene. First, we'll need to set up some additional geometry so that we get a surface that the object can cast shadows on. We'll want to separate the background and foreground into multiple draw calls so that we can also easily use different transformations, and shaders, for them.

### 3.4.1 More geometry

Start with one of the objects from before (the icosahedron, the bunny, or the teapot). In addition, let's create a room in which the object lies. This room will be very similar to the cube you created in Lab 2. You can copy code from that, just making sure that the cube is big enough so that both the object and the camera are on the inside. The main additional difference here is that the normals of your cube should now point *inwards*, since we are inside the cube. You can set up a separate VAO (let's call it `vao_room`) and buffer objects (`vbo_room`, `ebo_room`) and push them to the GPU as you did before. Set up your new VAO the same way as before, after the VAO for the object is set up. When drawing, you will first bind the first VAO, then make a draw call (exactly as you did up to task 3.3); and then bind your new VAO and make another draw call. If you did Lab 1.8 (multiple cubes), you will recognise this.

You may now want to have a separate "model" matrix for the light source, so that you can rotate it with the keyboard, the same way as you do for the object. To make sure that the light source and the camera are where they should be, relative to each other, verify that your light position is defined in world coordinates and that you apply the inverse view matrix to the light position in your C++ file before you write it to the `uniform` light position variable in the shader.

Finally, since you are now doing two separate draw calls, you might also use two different shader programs (optional). To do so, compile and link a shader program (`shader_program2 = glCreateProgram();` etc) similarly to as you have done it before, and call `glUseProgram` with one or the other program before each draw call. Please note that when you set your uniforms, you need to set them once for each program. For example, you can use different BRDFs in your shader programs: one for the object (occluder) and one for the room (receiver).

```
glUseProgram( shader_program );
```



Figure 6: Bunny inside a room. *Left:* same shader. *Right:* different shaders (Lambertian BRDF for the room, Blinn-Phong for the bunny).

```
glUniformMatrix4fv(glGetUniformLocation(shader_program ↩
    ,"viewMatrix"), 1, GL_FALSE, glm::value_ptr(View));
//...
glDraw//...
glUseProgram( shader_program2 );
glUniformMatrix4fv(glGetUniformLocation(shader_program2 ↩
    ,"viewMatrix"), 1, GL_FALSE, glm::value_ptr(View));
glDraw//...
```

The result should look something like Figure 6.

### 3.4.2 Making a framebuffer for the shadow map

With the new geometry in place, we can start rendering and using a shadow map so that the object can cast shadows on the floor and walls.

The content of the shadow map will be written to a second framebuffer, which we can then read from and determine what is in shadow and what is not, before rendering the colours of the scene to the main framebuffer.

Create a new framebuffer like so:

```
unsigned int depthMapFBO;
glGenFramebuffers(1, &depthMapFBO);
glBindFramebuffer(GL_FRAMEBUFFER, depthMapFBO);
```

Before, we have used OpenGL's default depth buffering just by calling `glEnable (GL_DEPTH_TEST);`. Now we will have to make our own depth buffer to generate the shadow map. Create a 2D texture that will be used as the depth buffer of the new frame buffer. This will be similar to the texture in Lab 3.1. Generate and bind the texture like before. Remember `glTexImage2D` and the mipmap parameters from before. Again, we have a `GL_TEXTURE_2D` target, level 0, but *internalformat* and *format* is now `GL_DEPTH_COMPONENT` because we only care about depth values. The *type* should now be `GL_FLOAT` since we will be computing floats and store here, rather than reading ints from an image file. You can set the width and height to 1000, for example. Finally, *data* should be NULL. (We have no pointer to image data. Filling the texture will happen as soon as we render to our framebuffer.)

Before we can use our new framebuffer, we need to *attach* another buffer to it (a color, depth, or stencil buffer). With the generated depth texture we can attach it as the framebuffer's depth buffer using a call to `glFramebufferTexture2D`. The *attachment* parameter should be `GL_DEPTH_ATTACHMENT`, and the *texture* parameter should be the name of the texture you generated with `glGenTextures` above.

We only need the depth information for the shadow map so there is really no need for a color buffer. However, a framebuffer object is not *complete* without a color buffer, so we need to explicitly tell OpenGL we're not going to render any color data. Do this by setting both the read and draw buffer to GL_NONE with glDrawBuffer and glReadbuffer.

```
  glDrawBuffer(GL_NONE);
  glReadBuffer(GL_NONE);
```

Verify that your framebuffer setup is complete:

```
if( glCheckFramebufferStatus(GL_FRAMEBUFFER) == ↩
    GL_FRAMEBUFFER_COMPLETE )
  std::cout << "framebuffer for shadow mapping complete";
```

Now we can unbind the framebuffer, since its setup is done.

```
  glBindFramebuffer( GL_FRAMEBUFFER, 0 );
```

### 3.4.3 Rendering the shadow map

Now it is time to write values to the shadow map, which we will use in Section 3.4.4 to determine what is shadowed. While the steps above are done once at the setup of the program, the rest must be done for each iteration in the main draw loop.

```
  glViewport(0, 0, shadow_width, shadow_height);
  glBindFramebuffer(GL_FRAMEBUFFER, depthMapFBO); // render to our ↩
      framebuffer instead of the default
  glClear(GL_DEPTH_BUFFER_BIT);
```

All the previous code skeletons have included a call to `glViewPort` to update the resolution when the window size changes. Here, we need to call `glViewPort` to set the right size of the shadow map. It should match the size you set for `glTexImage2D` above.

Next, we will set up the transformations needed to render the scene from the light's point of view. This will be very similar to the usual ModelViewProjection matrix, but the View matrix will be the inverse of the light's "view matrix" instead. The function `glm::lookAt` is convenient for generating a "lightView" matrix:

```
glm::mat4 lightView =
  glm::lookAt(glm::vec3(-2.0f, 4.0f, -1.0f),  // light pos in world
              glm::vec3( 0.0f, 0.0f,  0.0f),  // light "look-at" point
              glm::vec3( 0.0f, 1.0f,  0.0f)); // up vector
```

The code above generates an *inverse view matrix* for a given viewpoint, looking towards the origin, with the camera's *up* direction along the $y$ axis. This matrix should be multiplied as-is with a model and projection matrix.

*Question 1* How would you do this without the `lookAt` function?

The Projection matrix for the light specifies how wide the light frustum is. You can make this matrix similar to your camera projection matrix, just ensure that the aspect ratio matches with the aspect ratio of your shadow map. Try playing with different angles, and near and far limits.

With the light's view and projection matrices, we can render the scene as usual as long as we give each shader the light-space equivalents of the projection and view matrices. (Try rendering the scene using the light view & projection matrices just to verify that your transformations are set up correctly, but then revert back.)

Now, when creating the shadow map we only care about depth values and not all the potentially expensive fragment (lighting) calculations. Therefore you should use a separate shader program for this render pass, which uses the same vertex shader, but a trivial fragment shader that just contains an empty `void main` function.

Finally, after the draw call(s), reset to the main framebuffer by doing

```
glBindFramebuffer( GL_FRAMEBUFFER, 0 ); // unbind
glViewport(0, 0, w_width, w_height);
```

Summarising, rendering to the shadow map (after having set up the framebuffer) consists of the following steps.

1. Set viewport to the size of the shadow map
2. Bind the shadow map framebuffer
3. Compute light's view matrix, construct a ViewProjection matrix as seen from the light source and update the uniform in the shader program
4. For each object
   - Bind VAO for the object
   - Set its model matrix
   - Draw to shadow map
5. Unbind the shadow map framebuffer (doing so switches to the default)
6. Set viewport to width and height of window
7. Compute camera's ViewProjection matrix and update the shader program
8. Draw the scene

### 3.4.4 Using the shadow map

Now that the shadow map has been created, we can start rendering the actual shadows. Before we get to the shadow check inside the fragment shader, we also need to to a little more work in the vertex shader. In your main vertex shader, you should compute the vertex position in *view space* as before, and also the vertex position in *light space*, using the light's view and projection matrices. So your vertex shader should now additionaly provide an `out vec3 pos_light`.

When we output a clip-space vertex position to `gl_Position` in the vertex shader, OpenGL automatically does *homogenisation*; i e, divides the $x$, $y$ and $z$ component by the vector's $w$ component in order to get perspective. Since `pos_light` is not passed to the fragment shader through `gl_Position`, we have to do this perspective divide ourselves, in the fragment shader:

```
vec3 pos_light_perspective = pos_light.xyz / pos_light.w;
```

This returns the fragment's light-space position in the "NDC" range $[-1, 1]$. However, the depth from the depth map is in the range $[0, 1]$. Since we want to use `pos_light_perspective` to sample from the depth map, let's transform the NDC coordinates to the range $[0, 1]$:

```
pos_light_perspective = pos_light_perspective * 0.5 + 0.5;
```

Finally, check the depth value stored in the shadow map at the current fragment's $xy$ position as rendered from the light source

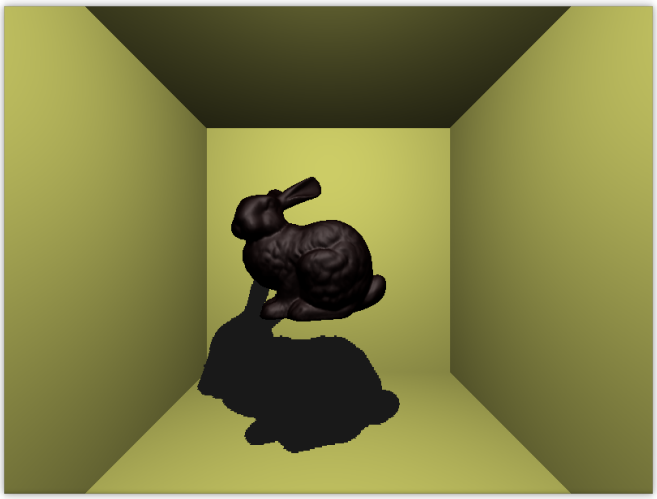


Figure 7: Bunny in a room, with shadow mapping.

```
float closestDepth = texture(shadowMap, pos_light_perspective.xy).r;
```

and compare with the fragment's distance to the light source.

```
float currentDepth = pos_light_perspective.z;
```

Output a dark colour if the current distance is greater than the one in the shadow map, and the normal material colour otherwise.

#### 3.4.5 Fight z fighting

Following the instructions from above, your shadows should be very noisy, because of $z$ fighting. Add a small bias to offset the shadow test, to make it cleaner.

The final result should look something like Figure 7.

+1 point

## 3.5 Additional questions

*Question 2*
- Why is $z$ fighting an issue for shadow mapping, but not for environment mapping?

*Question 3*
- Why can't you just use ordinary bilinear interpolation for countering aliasing effects in shadow mapping?

*Question 4*
- Give an example of a scene or an application where shadow volumes are preferable to shadow mapping.

*Question 5*
- Discuss examples of cases where displacement mapping would be better than normal mapping, and vice versa.

*Question 6*
- What would the normal map in Task 3.3 (that is, the blue long image with numbers in it) look like if the normals where instead specified as 3D vectors in *object* space instead of *tangent* space?